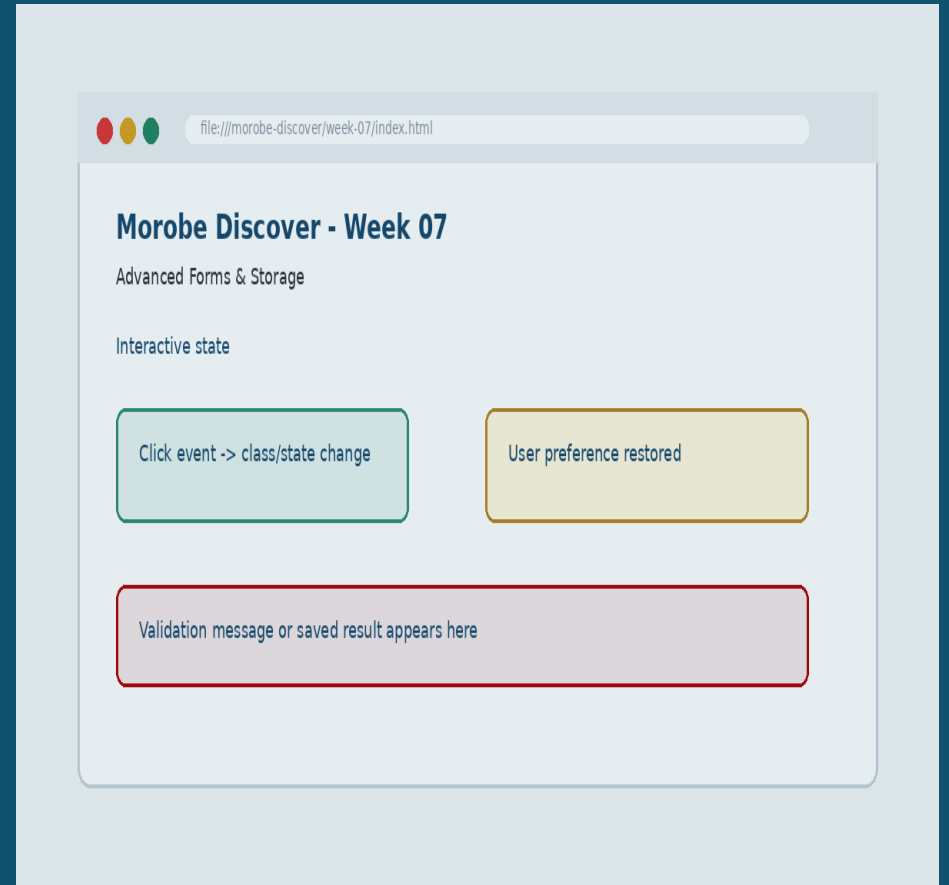


Week 07

Advanced Forms & Storage

Implement client-side validation and persist lightweight user choices between browser sessions.



Learning outcomes

Implement client-side validation and persist lightweight user choices between browser sessions.

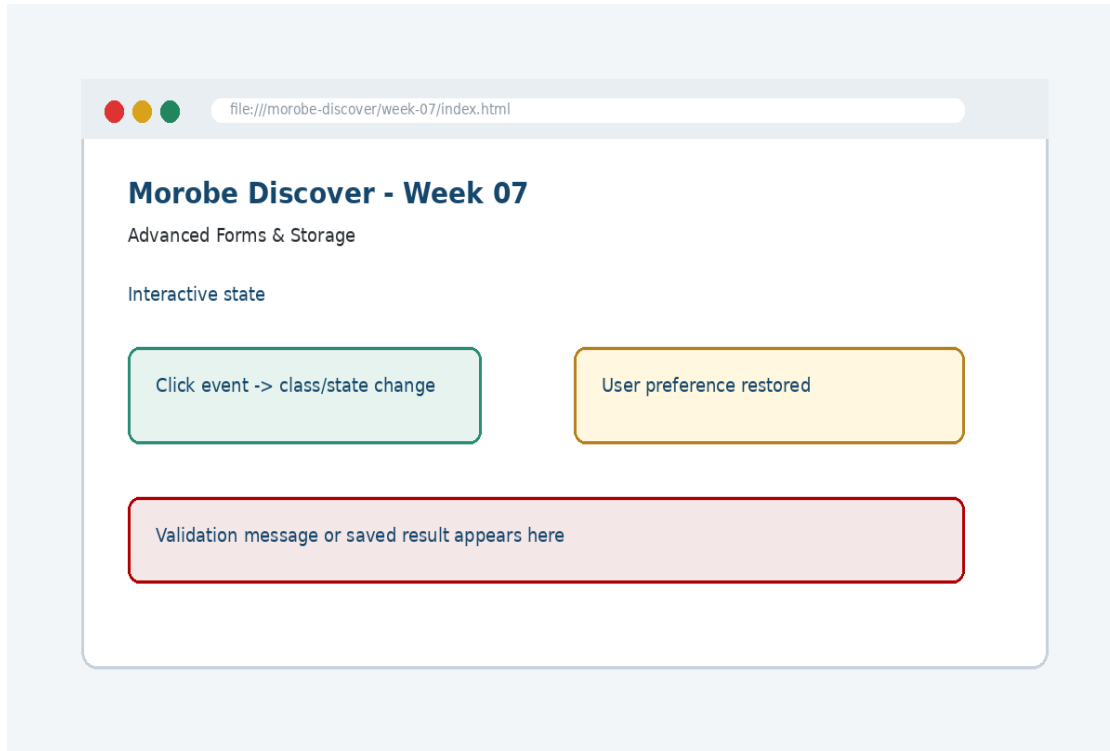
Explain the core programming/design concept for this week

Read and modify key code patterns

Test the weekly project increment

Prepare evidence for GitHub submission

Today's problem



Why it matters

Validate close to the user and store only lightweight, non-sensitive values in browser storage.

Project before -> after

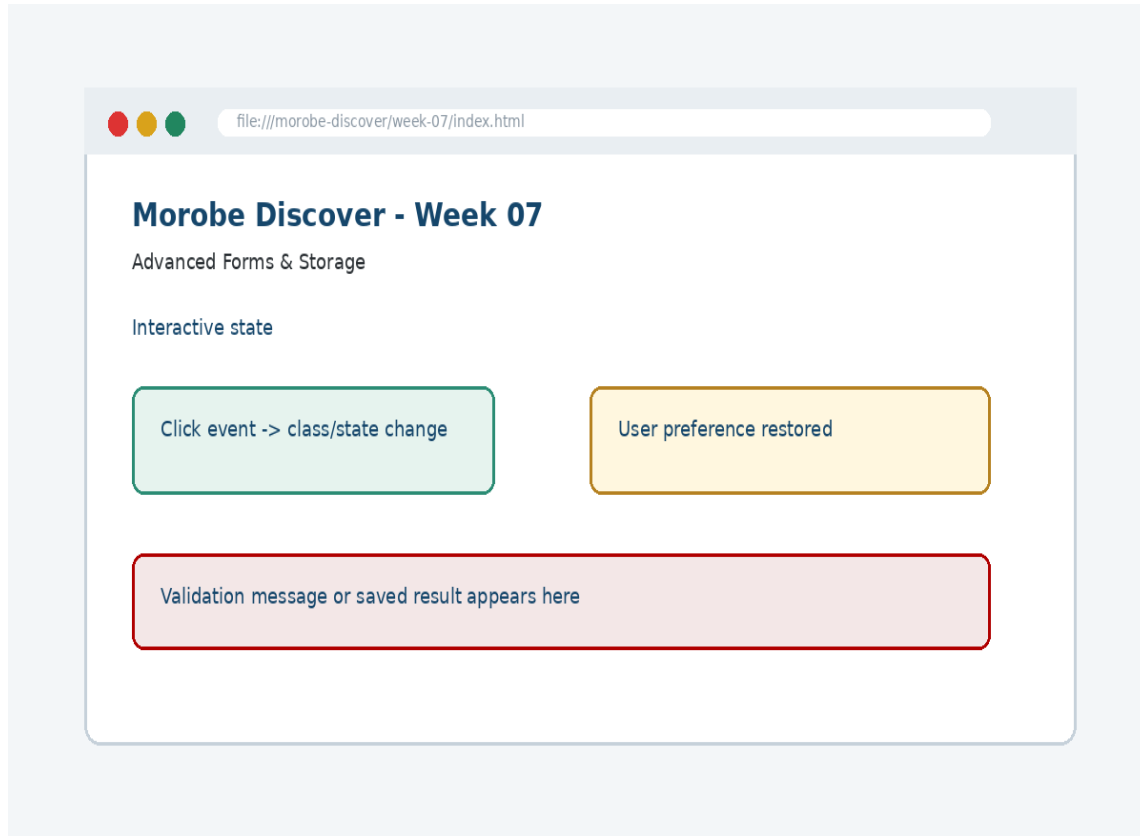
Before Week 07

Week 06 added DOM interaction and state changes.

After Week 07

The Plan a Trip form gives clearer feedback and remembers safe lightweight preferences.

What we build this week



Add custom validation messages
Prevent submission when required logic fails
Save theme choice or favourite destination
IDs
Recover saved values on page load
Display clear feedback to users

Core principle

Validate close to the user and store only lightweight, non-sensitive values in browser storage.

Keep in mind

A good web developer can point to the code, the browser result, and the user reason behind the decision.

Key concepts

Concept 1

Client-side validation improves feedback but does not replace server validation

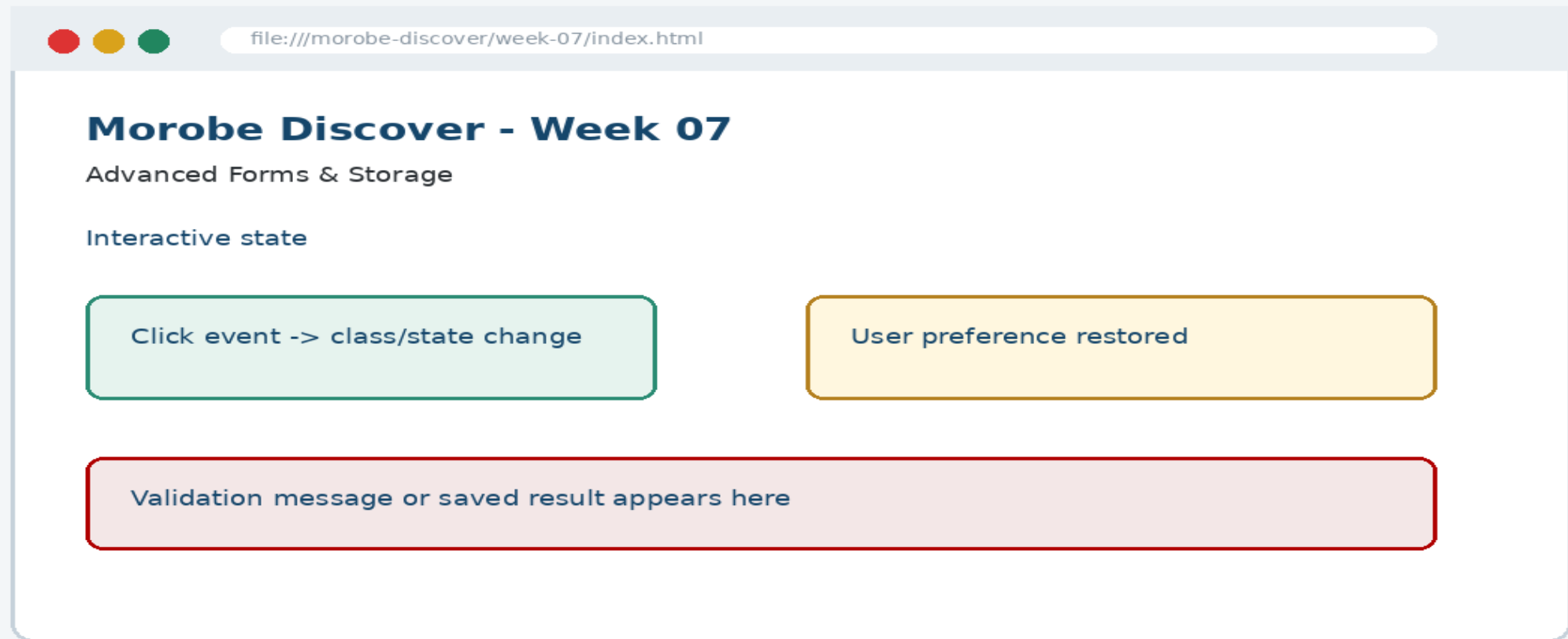
Concept 2

localStorage stores strings across sessions

Concept 3

Stored data should be limited and safe

Project anatomy



Key code pattern A

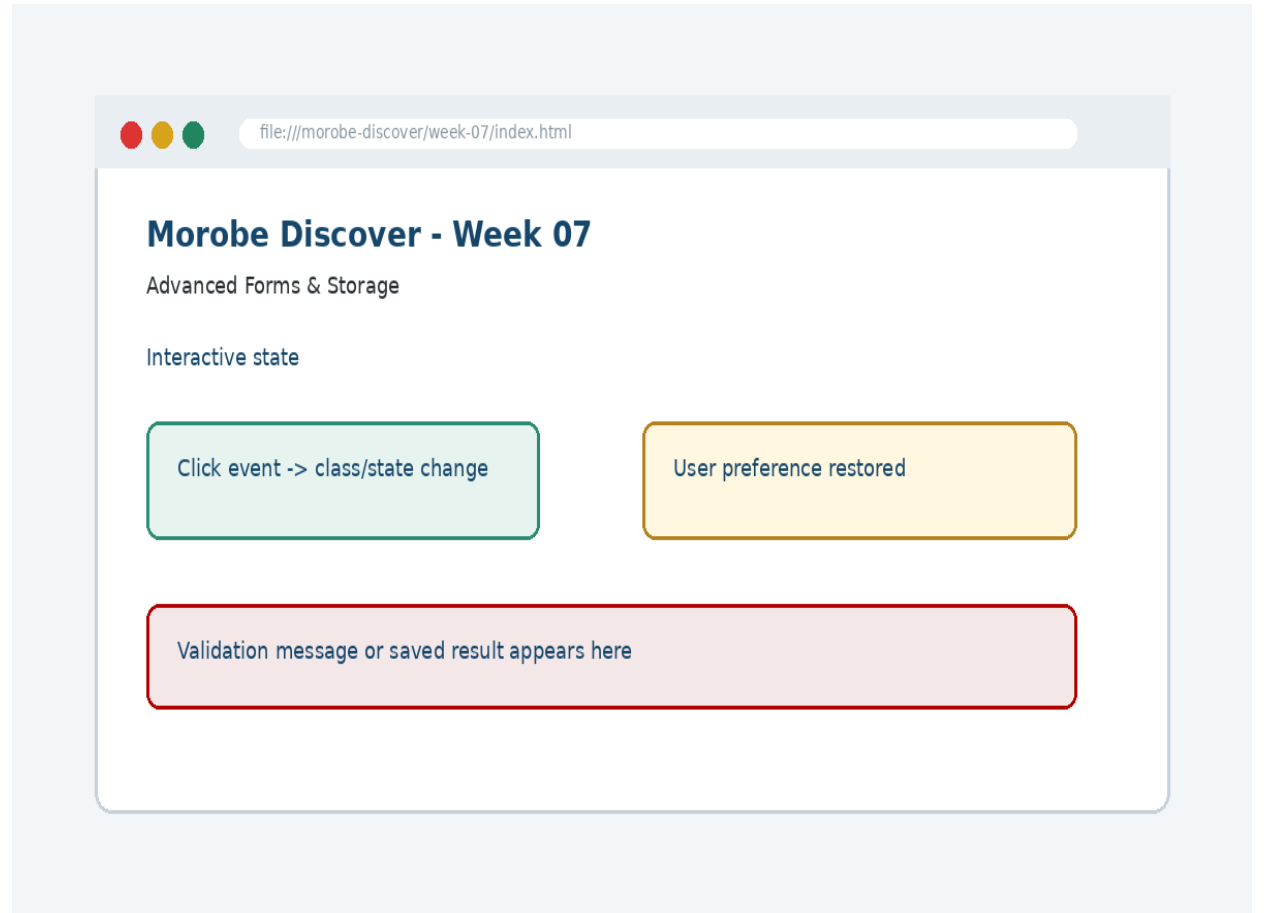
```
if (days < 1) {  
    showError("Trip length must be at least one  
day.");  
    return;  
}
```

Explain it

Which file contains this code?
Which line creates the visible change?
What should appear in the browser?

Code -> visual result

```
if (days < 1) {  
  showError("Trip length must be at  
  least one day.");  
  return;  
}
```



What changed?

Project file changed for Advanced Forms & Storage

The browser output is now closer to the required weekly evidence

The change supports a user task or quality requirement

The change can be tested and committed

Key code pattern B

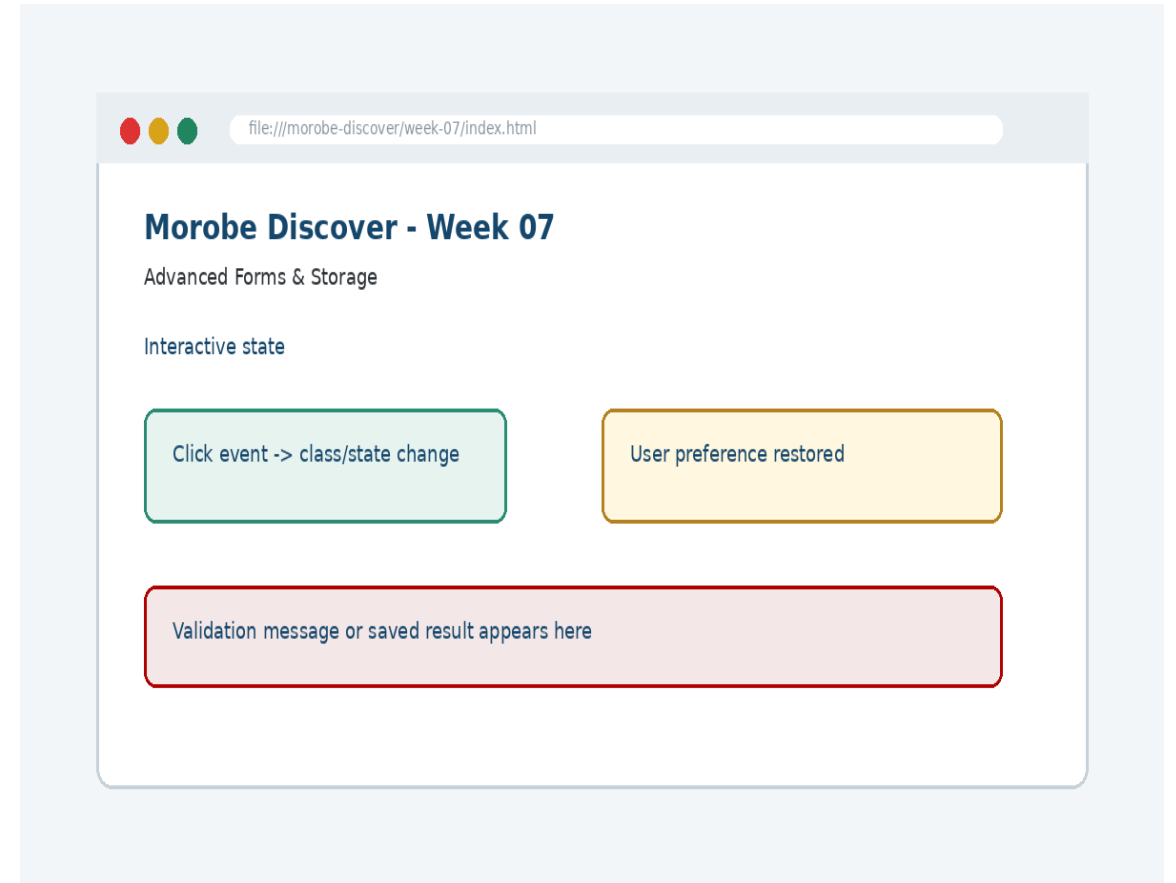
```
localStorage.setItem("theme", selectedTheme);  
const savedTheme = localStorage.getItem("theme");
```

Why this pattern is needed

The second pattern usually applies the same idea to a different component, state, layout or data flow.

Apply it to Morobe Discover

Add custom validation messages
Prevent submission when required logic fails
Save theme choice or favourite destination IDs
Recover saved values on page load



Compare the approaches

Weak approach

Memorise the definition
Copy code without understanding
No browser evidence
No test note

Professional approach

Explain the concept
Apply the correct code pattern
Inspect the result
Commit and describe the evidence

Common mistake

Mistake

The page appears unchanged, but the student assumes the feature is complete.

Correction

Check the correct file, refresh the browser, inspect DevTools, and verify the expected output.

Why does this fail?

```
// Broken checkpoint
// File changed, but the browser result does not
change.
// Possible causes:
// - wrong file
// - file not linked
// - syntax error
// - browser not refreshed
```

Class question

Which cause would you check first, and why?

Fix the problem

- Confirm the browser is opening the correct file
- Check links to CSS/JS/data files
- Inspect the console for errors
- Use a small working example
- Test again before committing

Class activity

Activity - 7 minutes

Decide which values belong in localStorage and which should never be stored there.

Work in pairs

Point to the project evidence

Prepare one answer to share

Implementation steps 1-3

- Add custom validation messages
- Prevent submission when required logic fails
- Save theme choice or favourite destination IDs

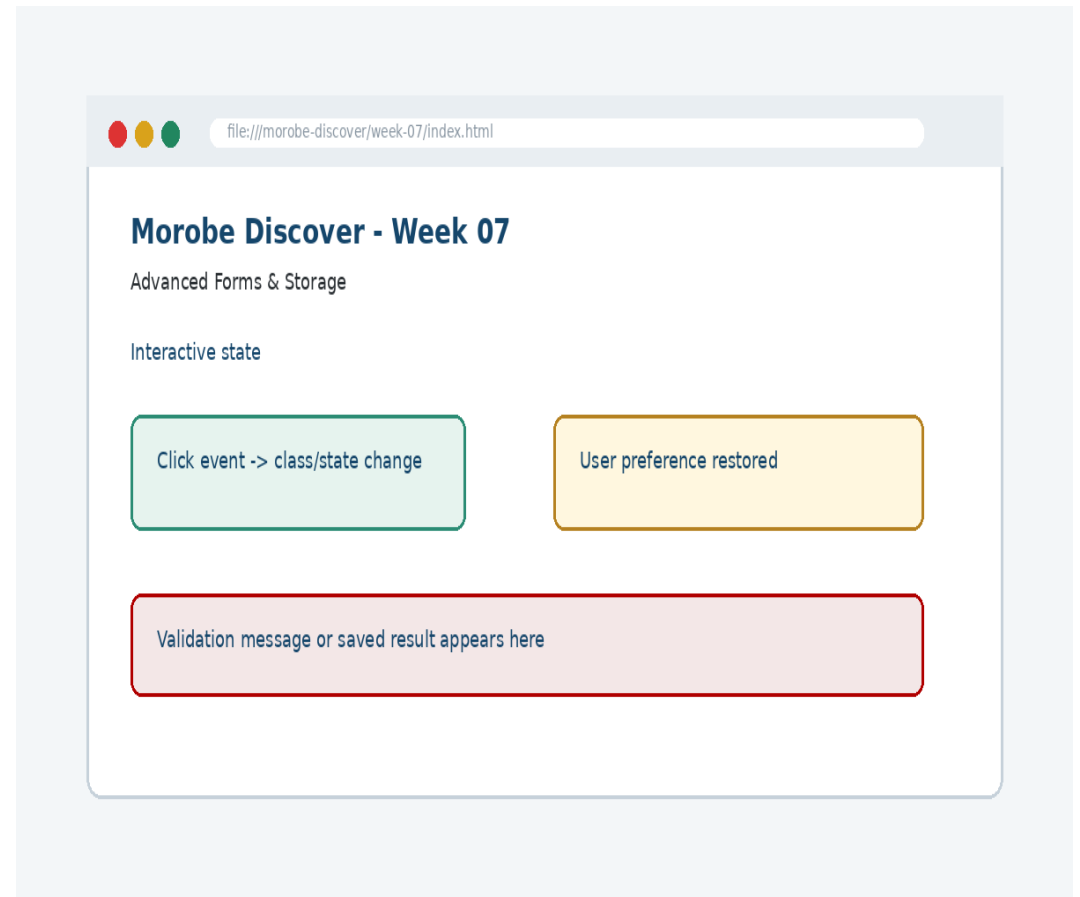
Implementation steps 4-5

Recover saved values on page load
Display clear feedback to users

LIVE DEMO

Demo checklist

1. Open the current project
2. Locate the affected file
3. Add or modify the key code
4. Refresh the browser
5. Inspect the result
6. Explain the change



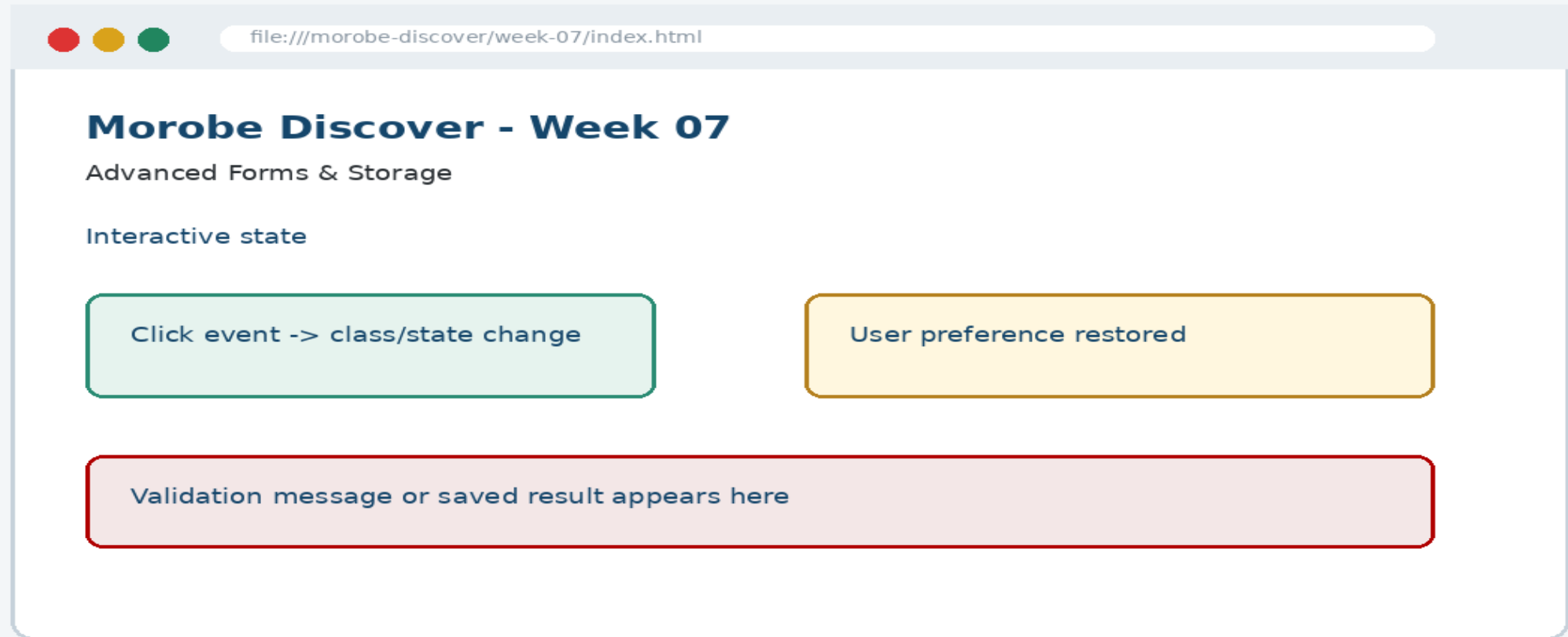
Predict the output

```
localStorage.setItem("theme", selectedTheme);  
const savedTheme =  
localStorage.getItem("theme");
```

Prediction prompt

What should change on screen when this code is applied correctly?

Browser result



Testing the feature

Submit invalid values

Refresh the browser and check saved preferences

Clear storage and retest default state

Check storage does not contain private data

Inspect with browser tools

Inspection path

Right click -> Inspect
Elements panel -> structure/classes
Console -> JavaScript errors
Network/Application -> data/storage when relevant

Evidence to save

Screenshot
Short test note
Changed files
Git commit message

Git evidence

```
git add .  
git commit -m "Week 07 Advanced Forms & Storage  
project increment"
```

Why it matters

Git history shows progression from Week 01 to final deployment. It is part of the assessment evidence.

Complete source code

Repository:
`https://github.com/YOUR-USERNAME/is229-morobe-discover`

Weekly folder:
`/week-07/`

Complete project:
`/complete-project/`

Reminder

Do not paste a whole file into a slide. Use the repository for complete code.

Mini challenge

Challenge

Modify one part of the Week 07 implementation and explain what changed in the browser.

State the file edited

State the line/section changed

Show the before and after result

What counts as evidence?

Technical evidence

Changed code files
Browser screenshot
Test note
Git commit

Understanding evidence

Code explanation
Why this method fits
Connection to previous week
Connection to next week

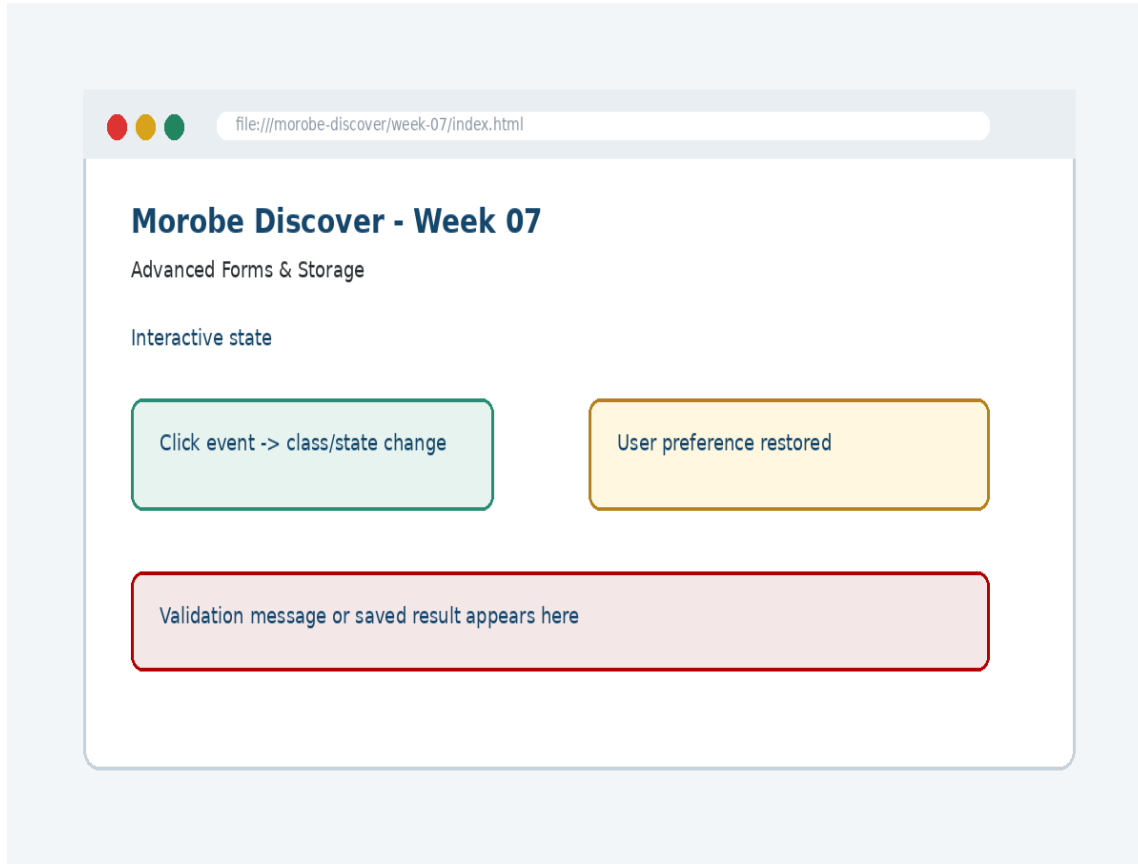
Project lab handoff

- Add custom validation messages
- Prevent submission when required logic fails
- Save theme choice or favourite destination IDs
- Recover saved values on page load
- Display clear feedback to users

Definition of done

Week 07 feature is implemented
Project still preserves earlier week features
Browser result matches the expected output
Test note and screenshot are saved
Git commit message is meaningful

What we built today



The Plan a Trip form gives clearer feedback and remembers safe lightweight preferences.
Evidence: Validation logic and user preference storage using localStorage
Technology: JavaScript / Web Storage API

Lecture summary

Validate close to the user and store only lightweight, non-sensitive values in browser storage.

Client-side validation improves feedback but does not replace server validation

localStorage stores strings across sessions

Testing and Git are part of development, not extras

Prepare for next class

Finish this week's lab before the next class
Read the next topic in advance
Bring the current project ready to extend
Be ready to explain one code decision

Exit question

How did Week 07 improve Morobe Discover, and what will the next week build on?